```
// STARS AI — petit serveur pour Render.
// - Sert la page (public/index.html)
// - Inscription : un code à 6 chiffres est envoyé par e-mail via un script
Google (depuis le Gmail du propriétaire)
// - Relaie les messages vers l'IA : Claude (ANTHROPIC_API_KEY) ou Gemini
(GEMINI_API_KEY)
// Aucune dépendance : Node 20+ suffit.

import http from "node:http";
import crypto from "node:crypto";
import { readFile } from "node:fs/promises";
import { fileURLToPath } from "node:url";
import path from "node:path";

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const PORT = Number(process.env.PORT) || 3000;
const env = k => (process.env[k] || "").trim();

// ---------- Réglages (modifiables dans Render > Environment) ----------
const ANTHROPIC_API_KEY = env("ANTHROPIC_API_KEY");
const GEMINI_API_KEY = env("GEMINI_API_KEY");
const CLAUDE_MODEL = env("CLAUDE_MODEL") || "claude-haiku-5-5";
const GEMINI_MODEL = env("GEMINI_MODEL") || "gemini-3-flash-preview";
const MAX_TOKENS = Number(env("MAX_TOKENS")) || 8192; // les corrections
d'exercices peuvent être longues
const PER_MINUTE = Number(env("PER_IP_PER_MINUTE")) || 15;          //
messages / minute / personne
const DAILY_LIMIT = Number(env("DAILY_LIMIT")) || 500;              //
messages / jour pour tout le site
const FIRST_BYTE_TIMEOUT_MS = Number(env("FIRST_BYTE_TIMEOUT_MS")) || 20000;
const ANTHROPIC_BASE_URL = env("ANTHROPIC_BASE_URL") ||
"https://api.anthropic.com";
const GEMINI_BASE_URL = env("GEMINI_BASE_URL") ||
"https://generativelanguage.googleapis.com";

// Inscription
const MAIL_WEBHOOK_URL = env("MAIL_WEBHOOK_URL");       // URL /exec du
script Google
const MAIL_WEBHOOK_SECRET = env("MAIL_WEBHOOK_SECRET"); // même valeur que
SECRET dans le script
const AUTH_REQUIRED = env("AUTH_REQUIRED") ? env("AUTH_REQUIRED") !== "0" :
!!MAIL_WEBHOOK_URL;
```

```
const SESSION_SECRET = env("SESSION_SECRET") ||
crypto.randomBytes(32).toString("hex");
const SESSION_DAYS = Number(env("SESSION_DAYS")) || 30;
const CODE_MINUTES = 10;
const CODE_ATTEMPTS = 5;
// Liste des comptes (Supabase) : sépare « S'inscrire » et « Se connecter »
const SUPABASE_URL = env("SUPABASE_URL").replace(/\/+$/, "");
const SUPABASE_KEY = env("SUPABASE_KEY");
const MEMBERS_SECRET = env("MEMBERS_SECRET");
const MEMBERS = !!(SUPABASE_URL && SUPABASE_KEY && MEMBERS_SECRET);
// Premium (paiement mobile money validé à la main dans /admin)
const FREE_DAILY = Number(env("FREE_DAILY_MESSAGES")) || 25;
const PREMIUM_PRICE = Number(env("PREMIUM_PRICE")) || 1000;
// Toucher un service ouvre directement l'appli (Wave) ou le menu USSD
(Orange #144#, MTN *133#, Moov *155#).
const WAVE_LINK = /^https:\/\/([a-z0-
9-]+\.)*wave\.com\//i.test(env("PAY_WAVE_LINK")) ? env("PAY_WAVE_LINK") : "";
const WAVE_STORE = "https://play.google.com/store/apps/details?
id=com.wave.personal";
// Lien marchand Wave (pay.wave.com/m/…) : ouvre l'appli Wave directement,
montant déjà rempli (on y met le prix du Premium).
const WAVE_MERCHANT = (() => {
  if (!WAVE_LINK) return "";
  const u = new URL(WAVE_LINK);
  if (u.hostname !== "pay.wave.com" || !u.pathname.startsWith("/m/")) return
"";
  u.searchParams.set("amount", String(PREMIUM_PRICE));
  return u.toString();
})();
// Le lien du code QR Wave (qr.wave.com) n'est pas un site web : on le confie
directement à l'appli Wave (Android).
// Si l'appli ne sait pas l'ouvrir, Android affiche la page Play Store de
Wave (bouton « Ouvrir »).
function waveIntent(link) {
  const tail = ";S.browser_fallback_url=" + encodeURIComponent(WAVE_STORE) +
";end";
  if (!link) return
"intent://#Intent;package=com.wave.personal;action=android.intent.action.MAIN
;category=android.intent.category.LAUNCHER" + tail;
  const u = new URL(link);
  return "intent://" + u.host + u.pathname + u.search +
"#Intent;scheme=https;package=com.wave.personal;action=android.intent.action.
VIEW" + tail;
}
const PAY_OPEN = {
```

```
  wave: WAVE_MERCHANT || waveIntent(WAVE_LINK),
  orange: "tel:%23144%23",
  mtn: "tel:*133%23",
  moov: "tel:*155%23"
};
const PAY_HOW = {
  wave: WAVE_MERCHANT
    ? "Wave s'ouvre directement avec le bon montant : confirme le paiement,
puis reviens ici avec la référence du reçu Wave."
    : "Wave s'ouvre (si le Play Store s'affiche, touche « Ouvrir »). Envoie
le montant au numéro copié, puis reviens ici.",
  orange: "Ton téléphone compose #144# : choisis « Transfert d'argent »,
colle le numéro et entre le montant.",
  mtn: "Ton téléphone compose *133# : choisis « Transfert », colle le numéro
et entre le montant.",
  moov: "Ton téléphone compose *155# : choisis « Transfert », colle le numéro
et entre le montant."
};
const PAY_OPEN_LABEL = { wave: WAVE_MERCHANT ? "Payer avec Wave" : "Ouvrir
Wave", orange: "Composer #144#", mtn: "Composer *133#", moov: "Composer
*155#" };
// Wave Checkout (API Wave Business) : le Premium s'active tout seul dès que
Wave confirme le paiement.
// Inactif tant que WAVE_API_KEY est vide (on garde alors le paiement Wave
validé à la main).
const WAVE_API_KEY = env("WAVE_API_KEY");
const WAVE_WEBHOOK_SECRET = env("WAVE_WEBHOOK_SECRET");
const WAVE_API_URL = (env("WAVE_API_URL") ||
"https://api.wave.com").replace(/\/+$/, "");
const WAVE_AUTO = !!(WAVE_API_KEY && MEMBERS);
const PAY_METHODS = [["wave", "Wave", env("PAY_WAVE") || (WAVE_MERCHANT ||
WAVE_AUTO ? "—" : "")], ["orange", "Orange Money", env("PAY_ORANGE")],
["mtn", "MTN MoMo", env("PAY_MTN")], ["moov", "Moov Money", env("PAY_MOOV")]]
  .filter(m => m[2]).map(([id, label, number]) => {
    const m = { id, label, number, open: PAY_OPEN[id], openLabel:
PAY_OPEN_LABEL[id], how: PAY_HOW[id], direct: id === "wave" &&
!!WAVE_MERCHANT };
    if (id !== "wave" || !WAVE_AUTO) return m;
    // Si Wave ne répond pas, la page revient au paiement Wave habituel
(validé à la main).
    const manual = WAVE_MERCHANT || env("PAY_WAVE") ? { number: m.number,
open: m.open, openLabel: m.openLabel, how: m.how, direct: m.direct } : null;
    return { id, label, number: "—", auto: true, direct: true, open: null,
openLabel: "Payer avec Wave",
      how: "Wave s'ouvre avec le bon montant : confirme le paiement, puis
```

```
reviens ici. Ton Premium s'active tout seul, sans attendre.", manual };
  });
const CONTACT_WHATSAPP = env("CONTACT_WHATSAPP").replace(/[^\d]/g, "");
const CONTACT_EMAIL = env("CONTACT_EMAIL") || "yvanemambo@gmail.com";
const ADMIN_EMAILS = new
Set(env("ADMIN_EMAILS").toLowerCase().split(/[,\s]+/).filter(Boolean));
const isAdmin = u => !!u && ADMIN_EMAILS.has(u.email);

// Notifications sur le téléphone du propriétaire (Web Push sans contenu : le
téléphone vient chercher le détail dans /api/admin/overview).
// Les clés sont créées par le serveur au premier besoin et gardées dans
Supabase (table verrouillée stars_config) : personne n'a à les manipuler.
const VAPID_SUBJECT = env("VAPID_SUBJECT") || "mailto:" + CONTACT_EMAIL;
let vapidKey = null, VAPID_PUBLIC = "";
async function loadVapid() {
  if (vapidKey) return true;
  if (!MEMBERS) return false;
  try {
    const { publicKey, privateKey } = crypto.generateKeyPairSync("ec", {
namedCurve: "prime256v1" });
    const jwk = publicKey.export({ format: "jwk" });
    const proposal = JSON.stringify({
      pub: Buffer.concat([Buffer.from([4]), Buffer.from(jwk.x, "base64url"),
Buffer.from(jwk.y, "base64url")]).toString("base64url"),
      priv: privateKey.export({ format: "der", type: "pkcs8"
}).toString("base64url")
    });
    const stored = JSON.parse(await rpc("stars_config_get_or_init", { p_key:
"vapid", p_value: proposal }));
    vapidKey = crypto.createPrivateKey({ key: Buffer.from(stored.priv,
"base64url"), format: "der", type: "pkcs8" });
    VAPID_PUBLIC = stored.pub;
    return true;
  } catch (e) { console.warn("[push] clés de notification indisponibles :",
e.message); return false; }
}
const PUSH_HOSTS =
/^(fcm\.googleapis\.com|android\.googleapis\.com|updates\.push\.services\.moz
illa\.com|[a-z0-9.-]+\.push\.apple\.com|[a-z0-
9.-]+\.notify\.windows\.com)$/i;
function pushEndpointOk(u) {
  try {
    const x = new URL(u);
    if (env("PUSH_ALLOW_ANY") === "1") return /^https?:$/.test(x.protocol);
// tests locaux uniquement
```

```
    return x.protocol === "https:" && PUSH_HOSTS.test(x.hostname);
  }
  catch { return false; }
}
function vapidJwt(aud) {
  const enc = o => Buffer.from(JSON.stringify(o)).toString("base64url");
  const unsigned = enc({ typ: "JWT", alg: "ES256" }) + "." + enc({ aud, exp:
Math.floor(Date.now() / 1000) + 12 * 3600, sub: VAPID_SUBJECT });
  const sig = crypto.sign("sha256", Buffer.from(unsigned), { key: vapidKey,
dsaEncoding: "ieee-p1363" }).toString("base64url");
  return unsigned + "." + sig;
}
async function notifyAdmins(reason) {
  if (!MEMBERS || !(await loadVapid())) return;
  let subs = [];
  try { subs = (await rpc("stars_push_list", {})) || []; } catch (e) { return
console.warn("[push]", e.message); }
  for (const s of subs) {
    if (!ADMIN_EMAILS.has(s.email) || !pushEndpointOk(s.endpoint)) continue;
    try {
      const r = await fetch(s.endpoint, {
        method: "POST", body: "", signal: AbortSignal.timeout(10000),
        headers: { TTL: "86400", Urgency: "high", Authorization: `vapid
t=${vapidJwt(new URL(s.endpoint).origin)}, k=${VAPID_PUBLIC}` }
      });
      if (r.status === 404 || r.status === 410) await
rpc("stars_push_disable", { p_endpoint: s.endpoint, p_error: "HTTP " +
r.status });
      else if (r.status >= 400) console.warn(`[push] refusé HTTP ${r.status}
: ${(await r.text().catch(() => "")).slice(0, 200)}`);
      else console.log(`[push] notification envoyée (${reason})`);
    } catch (e) { console.warn("[push]", e.message); }
  }
}
if (!env("SESSION_SECRET")) console.warn("[auth] SESSION_SECRET absent : les
connexions seront perdues à chaque redémarrage.");

// Modèles de secours si le modèle choisi n'existe plus, est surchargé ou ne
répond pas.
const CLAUDE_FALLBACKS = ["claude-haiku-5-5", "claude-sonnet-5-5"];
const GEMINI_FALLBACKS = ["gemini-3-flash-preview", "gemini-3.1-flash-lite",
"gemini-flash-latest", "gemini-flash-lite-latest"];

const provider = ANTHROPIC_API_KEY ? "claude" : GEMINI_API_KEY ? "gemini" :
null;
```

```
const VOICE_RULES = `

MODE VOCAL — ta réponse va être lue à voix haute pendant un appel :
- Parle comme à l'oral, de façon naturelle et chaleureuse, en 1 à 4 phrases
courtes. Développe seulement si on te demande d'expliquer en détail.
- Pas de Markdown, pas de listes, pas de titres, pas d'emojis, pas de
tableaux, pas de code, pas de liens.
- Dis les maths avec des mots : « x au carré », « 3 sur 4 », « racine de 16
».
- Pour un exercice, explique une étape à la fois, simplement, puis demande si
l'élève veut la suite.
- Si tu n'as pas bien compris la phrase (la reconnaissance vocale peut se
tromper), demande gentiment de répéter.`;

// Guide des langages informatiques (fourni par Yvan Mambo). Il est ajouté
aux consignes
// seulement quand la conversation parle de programmation, pour garder les
autres réponses rapides.
const GUIDE_LANGAGES = await readFile(path.join(__dirname, "knowledge",
"langages.md"), "utf8")
  .catch(() => { console.warn("[guide] knowledge/langages.md introuvable");
return ""; });
if (GUIDE_LANGAGES) console.log(`[guide] Guide des langages chargé
(${GUIDE_LANGAGES.length} caractères).`);
const GUIDE_TRIGGER = /langages? (?:de programmation|informatiques?)|quels?
langages?|meilleurs? langages?|premier langage|langage
c\b|programm(?:ation|er|eur|euse|ez|ons)\b|programme
(?:informatique|python|en)|\bcod(?:er|age|eur|euse)s?\b|\b(?:du|ce|mon|ton)
code\b|code (?:source|informatique)|d[ée]velopp(?:eur|euse)|d[ée]veloppement
(?:web|mobile|logiciel|d'appli)|informatique|logiciel|algorithm|framework|fro
nt-?end|back-?end|bases? de donn[ée]es|site (?:web|internet)|appli(?:cation)?
s?
(?:mobile|android|iphone|ios)|cybers[ée]curit|blockchain|arduino|microcontr|p
ython|javascript|typescript|\bjs\b|\bjava\b|kotlin|\bswift\b|html|\bcss\b|\bs
ql\b|\bphp\b|c\+\+|c#|\brust\b|golang|\bruby\b|\bdart\b|flutter|\breact\b|nod
e\.?
js|\bperl\b|\blua\b|haskell|cobol|fortran|assembleur|\bbash\b|powershell|soli
dity|matlab|scratch/i;
function guideFor(messages) {
  if (!GUIDE_LANGAGES) return "";
  const recent = messages.filter(m => m.role === "user").slice(-3).map(m =>
m.content).join("\n");
  return GUIDE_TRIGGER.test(recent) ? GUIDE_LANGAGES : "";
}
```

```
const GUIDE_INTRO = `

Guide de référence sur les langages informatiques (document fourni par Yvan Mambo, le créateur de STARS AI). Sers-t'en quand la question porte sur les langages de programmation : à quoi sert un langage, son année, ses usages, lequel choisir, comment apprendre. Réponds à la question posée sans recopier le guide en entier ; complète avec tes connaissances si le guide ne suffit pas.

<guide>
`;

function rules(name, voice, guide) {
  const tech = provider === "gemini"
    ? "STARS AI est propulsé par Gemini, un modèle de Google"
    : "STARS AI est propulsé par Claude, un modèle d'Anthropic";
  return `Tu es STARS AI, un assistant d'intelligence artificielle généraliste, chaleureux, clair et précis.
- Réponds dans la langue de l'utilisateur (français par défaut).
- Va droit au but ; développe quand la question le demande.
- Utilise du Markdown simple quand c'est utile : **gras**, listes, titres ###, blocs de code.
- Si on te demande qui tu es : tu es STARS AI.
- Si on te demande qui t'a créé, qui est ton créateur, qui t'a fabriqué ou qui a fait cette application : STARS AI a été créé par Yvan Mambo. Réponds simplement cela, sans parler de la technologie utilisée.
- Seulement si on te demande qui sont les parents ou la famille de ton créateur : Yvan Mambo est le fils de Mambo N'Gouan Marcelin (son père) et de Kassia Glahi Anastasie (sa mère). N'en parle pas si on ne te le demande pas.
- Seulement si on te demande précisément sur quel modèle ou quelle technologie d'IA tu fonctionnes, dis honnêtement que ${tech}, et que STARS AI a été conçu et créé par Yvan Mambo.
- Ne prétends jamais avoir fait une action que tu ne peux pas faire (naviguer sur internet, envoyer un message…).
- Tu peux voir les photos que l'utilisateur envoie (exercices, documents, schémas…).

Mode prof — quand on te donne un exercice (maths, physique-chimie, SVT, français, anglais, philo, etc.), tape ou en photo :
- Lis tout l'énoncé avec attention. Si une photo est floue ou qu'une partie est illisible, dis précisément ce que tu n'arrives pas à lire et demande une nouvelle photo, au lieu d'inventer.
- Explique comme un bon professeur, étape par étape : ce qu'on cherche, la méthode ou la règle utilisée, puis les calculs détaillés ligne par ligne.
- Donne le résultat final en **gras**, avec l'unité s'il y en a une. Traite
```

```
chaque question de l'exercice dans l'ordre (1, 2, 3…).
- Écris les formules en texte lisible avec des symboles : x², x³, √, ×, ÷, ±, ≤, ≥, ≠, ≈, π, Δ, →, °. N'utilise JAMAIS de LaTeX (pas de $…$, pas de \\frac, pas de \\sqrt). Écris les fractions comme 3/4 ou (x + 1)/(x − 2).
- Utilise des méthodes du niveau de l'élève (collège ou lycée, programme francophone) quand c'est possible.
- Si l'élève demande seulement un indice ou de l'aide pour commencer, donne un indice sans la solution complète.
- À la fin d'un exercice résolu, propose en une phrase un exercice similaire pour s'entraîner, sans donner sa réponse.${guide ? GUIDE_INTRO + guide.trim() + "\n</guide>" : ""}${name ? `\n\nL'utilisateur s'appelle ${name}. Tu peux l'appeler par son prénom de temps en temps, sans en abuser.` : ""}${voice ? VOICE_RULES : ""}`;
}

// Combine l'annulation par le visiteur et un délai maximum.
function withTimeout(signal, ms) {
  const t = AbortSignal.timeout(ms);
  return signal && AbortSignal.any ? AbortSignal.any([signal, t]) : t;
}

// ---------- Limites anti-abus (en mémoire) ----------
const buckets = new Map(); // clé -> [timestamps]
function allow(key, max, windowMs) {
  const now = Date.now();
  const list = (buckets.get(key) || []).filter(t => now - t < windowMs);
  if (list.length >= max) { buckets.set(key, list); return false; }
  list.push(now); buckets.set(key, list);
  return true;
}
setInterval(() => {
  const now = Date.now();
  for (const [k, list] of buckets) if (!list.some(t => now - t < 3_600_000))
buckets.delete(k);
  for (const [k, v] of attempts) if (v.exp < now) attempts.delete(k);
}, 5 * 60_000).unref();

// Plafond global du jour (protège le quota de la clé IA) : ne s'applique pas aux abonnés Premium.
let day = new Date().toISOString().slice(0, 10);
let dayCount = 0;
function checkSiteCap(premium) {
  const today = new Date().toISOString().slice(0, 10);
  if (today !== day) { day = today; dayCount = 0; }
  if (!premium && dayCount >= DAILY_LIMIT) return "daily_limit";
```

```
  dayCount++;
  return null;
}

// ---------- Jetons signés (code d'inscription + session) ----------
const b64 = s => Buffer.from(s).toString("base64url");
const unb64 = s => Buffer.from(s, "base64url").toString("utf8");
const sign = (kind, data) => crypto.createHmac("sha256",
SESSION_SECRET).update(kind + "|" + data).digest("base64url");
function safeEqual(a, b) {
  const x = Buffer.from(a), y = Buffer.from(b);
  return x.length === y.length && crypto.timingSafeEqual(x, y);
}
function makeToken(kind, payload, extra = "") {
  const body = b64(JSON.stringify(payload));
  return body + "." + sign(kind, body + "|" + extra);
}
function readToken(kind, token, extra = "") {
  if (typeof token !== "string" || !token.includes(".")) return null;
  const [body, sig] = token.split(".");
  if (!body || !sig || !safeEqual(sig, sign(kind, body + "|" + extra)))
return null;
  try { const p = JSON.parse(unb64(body)); return p && p.x > Date.now() ? p :
null; } catch { return null; }
}
function peekToken(token) {
  try { return JSON.parse(unb64(String(token).split(".")[0])); } catch {
return null; }
}

const attempts = new Map(); // nonce -> { n, exp, used }

function parseCookies(req) {
  const out = {};
  for (const part of (req.headers.cookie || "").split(";")) {
    const i = part.indexOf("=");
    if (i > 0) out[part.slice(0, i).trim()] = decodeURIComponent(part.slice(i
+ 1).trim());
  }
  return out;
}
function currentUser(req) {
  const p = readToken("sess", parseCookies(req).stars_session);
  return p ? { email: p.e, name: p.n } : null;
}
```

```
function sessionCookie(req, value, maxAgeSec) {
  const secure = req.headers["x-forwarded-proto"] === "https" ? "; Secure" :
"";
  return `stars_session=${encodeURIComponent(value)}; Path=/; HttpOnly;
SameSite=Lax; Max-Age=${maxAgeSec}${secure}`;
}

// ---------- Envoi via le script Google ----------
async function callMailer(payload) {
  if (!MAIL_WEBHOOK_URL) throw new Error("mail_not_configured");
  const res = await fetch(MAIL_WEBHOOK_URL, {
    method: "POST",
    redirect: "follow",
    signal: AbortSignal.timeout(25000),
    headers: { "content-type": "text/plain;charset=utf-8" },
    body: JSON.stringify({ secret: MAIL_WEBHOOK_SECRET, ...payload })
  });
  const text = await res.text();
  let data;
  try { data = JSON.parse(text); }
  catch { throw new Error(`réponse inattendue du script Google (HTTP
${res.status}) — vérifie « Qui a accès : Tout le monde » : ${text.slice(0,
160).replace(/\s+/g, " ")}`); }
  if (!data.ok) throw new Error(data.error || "mail_failed");
  return data;
}

// ---------- Comptes (Supabase) ----------
async function rpc(fn, args) {
  const headers = { "content-type": "application/json", apikey: SUPABASE_KEY
};
  if (SUPABASE_KEY.startsWith("eyJ")) headers.authorization = "Bearer " +
SUPABASE_KEY;
  const r = await fetch(`${SUPABASE_URL}/rest/v1/rpc/${fn}`, {
    method: "POST", headers, signal: AbortSignal.timeout(10000),
    body: JSON.stringify({ p_secret: MEMBERS_SECRET, ...args })
  });
  const text = await r.text();
  if (!r.ok) throw new Error(`Supabase ${fn} HTTP ${r.status} :
${text.slice(0, 200)}`);
  return text ? JSON.parse(text) : null;
}
async function findMember(email) {
  const rows = await rpc("stars_member_get", { p_email: email });
  return Array.isArray(rows) && rows[0] ? rows[0] : null;
```

```
}
const ensured = new Set();
function ensureMember(user) {
  if (!MEMBERS || ensured.has(user.email)) return;
  ensured.add(user.email);
  rpc("stars_member_ensure", { p_email: user.email, p_name: user.name || ""
})
    .catch(e => { ensured.delete(user.email); console.warn("[comptes]",
e.message); });
}

const EMAIL_RE = /^[^\s@<>"',;]+@[^\s@<>"',;]+\.[a-z]{2,}$/i;
function cleanName(s) {
  return String(s || "").replace(/[\u0000-\u001f\u007f<>]/g,
"").replace(/^[=+\-@]+/, "").trim().slice(0, 40);
}

// ---------- Mots de passe (seul un hachage scrypt est stocké) ----------
const PASSWORD_MIN = 8;
const passwordOk = pw => typeof pw === "string" && pw.length >= PASSWORD_MIN
&& pw.length <= 200;
const scryptAsync = (pw, salt, keylen, opts) => new Promise((ok, ko) =>
crypto.scrypt(pw, salt, keylen, opts, (e, k) => e ? ko(e) : ok(k)));
async function hashPassword(pw) {
  const salt = crypto.randomBytes(16);
  const key = await scryptAsync(pw.normalize("NFC"), salt, 32, { N: 16384, r:
8, p: 1 });
  return
`scrypt$16384$8$1$${salt.toString("base64")}$${key.toString("base64")}`;
}
async function checkPassword(pw, stored) {
  const parts = String(stored || "").split("$");
  if (parts.length !== 6 || parts[0] !== "scrypt") return false;
  const [, N, r, p, salt, hash] = parts;
  const expected = Buffer.from(hash, "base64");
  const key = await scryptAsync(String(pw).normalize("NFC"),
Buffer.from(salt, "base64"), expected.length, { N: +N, r: +r, p: +p, maxmem:
64 * 1024 * 1024 });
  return crypto.timingSafeEqual(key, expected);
}
// Le hachage voyage chiffré dans le jeton du code d'inscription (rien n'est
gardé en mémoire).
const ENC_KEY = crypto.createHash("sha256").update("enc|" +
SESSION_SECRET).digest();
function seal(text) {
```

```
  const iv = crypto.randomBytes(12);
  const c = crypto.createCipheriv("aes-256-gcm", ENC_KEY, iv);
  const out = Buffer.concat([c.update(text, "utf8"), c.final()]);
  return Buffer.concat([iv, c.getAuthTag(), out]).toString("base64url");
}
function unseal(b) {
  try {
    const buf = Buffer.from(b, "base64url");
    const d = crypto.createDecipheriv("aes-256-gcm", ENC_KEY, buf.subarray(0,
12));
    d.setAuthTag(buf.subarray(12, 28));
    return Buffer.concat([d.update(buf.subarray(28)),
d.final()]).toString("utf8");
  } catch { return null; }
}
const mask = email => email.replace(/^(.).*(@.*)$/, "$1***$2");

function openSession(req, res, email, name) {
  const sess = makeToken("sess", { e: email, n: name, x: Date.now() +
SESSION_DAYS * 86_400_000 });
  res.setHeader("set-cookie", sessionCookie(req, sess, SESSION_DAYS *
86_400));
}

// Connexion directe : Gmail + mot de passe
async function authPassword(req, res, ip) {
  if (!AUTH_REQUIRED) return json(res, 400, { code: "auth_disabled" });
  if (!MEMBERS) return json(res, 503, { code: "members_down" });
  const body = await readJson(req);
  const email = String(body?.email || "").trim().toLowerCase();
  const password = String(body?.password || "");
  if (email.length > 254 || !EMAIL_RE.test(email)) return json(res, 400, {
code: "bad_email" });
  if (!password) return json(res, 400, { code: "bad_password" });
  if (!allow("pw-ip:" + ip, 20, 15 * 60_000) || !allow("pw-mail:" + email, 8,
15 * 60_000)) return json(res, 429, { code: "too_many_passwords" });
  let member;
  try {
    const rows = await rpc("stars_member_auth", { p_email: email });
    member = Array.isArray(rows) && rows[0] ? rows[0] : null;
  } catch (e) { console.error("[comptes]", e.message); return json(res, 503,
{ code: "members_down" }); }
  if (!member) return json(res, 404, { code: "not_registered" });
  if (!member.password_hash) return json(res, 409, { code: "no_password" });
  if (!(await checkPassword(password, member.password_hash))) return
```

```
json(res, 401, { code: "bad_password" });
  const name = cleanName(member.name) || cleanName(email.split("@")[0]);
  rpc("stars_member_login", { p_email: email, p_name: name }).catch(e =>
console.warn("[comptes]", e.message));
  ensured.add(email);
  openSession(req, res, email, name);
  console.log(`[auth] connexion par mot de passe de ${mask(email)}`);
  json(res, 200, { email, name });
}

// Envoi d'un code : « signup » (inscription, avec mot de passe), « reset »
(mot de passe oublié), « login » (code seul)
async function authStart(req, res, ip) {
  if (!AUTH_REQUIRED) return json(res, 400, { code: "auth_disabled" });
  const body = await readJson(req);
  const mode = ["login", "reset"].includes(body?.mode) ? body.mode :
"signup";
  const email = String(body?.email || "").trim().toLowerCase();
  let name = cleanName(body?.name);
  if (email.length > 254 || !EMAIL_RE.test(email)) return json(res, 400, {
code: "bad_email" });
  if (mode === "signup" && !name) return json(res, 400, { code: "bad_name"
});
  if (mode === "signup" && MEMBERS && !passwordOk(body?.password)) return
json(res, 400, { code: "weak_password" });

  // Inscription : refusée si le compte existe déjà. Connexion / mot de passe
oublié : refusés si le compte n'existe pas.
  if (MEMBERS) {
    if (!allow("lookup-ip:" + ip, 20, 10 * 60_000)) return json(res, 429, {
code: "too_many_codes" });
    let member;
    try { member = await findMember(email); }
    catch (e) { console.error("[comptes]", e.message); return json(res, 503,
{ code: "members_down" }); }
    if (mode === "signup" && member) return json(res, 409, { code:
"already_registered" });
    if (mode !== "signup" && !member) return json(res, 404, { code:
"not_registered" });
    if (member) name = cleanName(member.name) || name;
  }
  if (!name) name = cleanName(email.split("@")[0]) || "ami";

  if (!allow("start-ip:" + ip, 6, 10 * 60_000) || !allow("start-mail:" +
email, 3, 10 * 60_000)) return json(res, 429, { code: "too_many_codes" });
```

```
  const code = String(crypto.randomInt(0, 1_000_000)).padStart(6, "0");
  const payload = { e: email, n: name, m: mode, x: Date.now() + CODE_MINUTES
* 60_000, j: crypto.randomBytes(9).toString("base64url") };
  if (mode === "signup" && MEMBERS) payload.h = seal(await
hashPassword(body.password));
  const token = makeToken("code", payload, code);
  try {
    await callMailer({ action: "send_code", email, name, code });
  } catch (e) {
    console.error("[auth] envoi du code impossible :", e.message);
    return json(res, 502, { code: e.message === "quota" ? "mail_quota" :
"mail_failed" });
  }
  console.log(`[auth] code envoyé à ${email.replace(/^(.).*(@.*)$/,
"$1***$2")}`);
  json(res, 200, { token, email });
}

async function authVerify(req, res, ip) {
  if (!allow("verify-ip:" + ip, 20, 10 * 60_000)) return json(res, 429, {
code: "too_many_attempts" });
  const body = await readJson(req);
  const token = String(body?.token || "");
  const code = String(body?.code || "").replace(/\D/g, "");
  const peek = peekToken(token);
  if (!peek?.j || !peek?.x) return json(res, 400, { code: "bad_token" });
  if (peek.x < Date.now()) return json(res, 400, { code: "code_expired" });
  // Mot de passe oublié : le nouveau mot de passe est vérifié avant de
consommer un essai.
  if (peek.m === "reset" && !passwordOk(body?.password)) return json(res,
400, { code: "weak_password" });
  const a = attempts.get(peek.j) || { n: 0, exp: peek.x, used: false };
  if (a.used) return json(res, 400, { code: "code_used" });
  if (a.n >= CODE_ATTEMPTS) return json(res, 429, { code: "too_many_attempts"
});
  const p = code.length === 6 ? readToken("code", token, code) : null;
  if (!p) {
    a.n++; attempts.set(peek.j, a);
    return json(res, 400, { code: "bad_code", left: Math.max(0, CODE_ATTEMPTS
- a.n) });
  }
  a.used = true; attempts.set(peek.j, a);
  if (MEMBERS) {
    let newHash = null;
```

```
    if (p.m === "reset") newHash = await hashPassword(body.password);
    else if (p.h) newHash = unseal(p.h);
    try {
      const rows = await rpc("stars_member_login", { p_email: p.e, p_name:
p.n });
      if (Array.isArray(rows) && rows[0]?.name) p.n = cleanName(rows[0].name)
|| p.n;
      if (rows?.[0]?.created) console.log(`[comptes] nouveau compte :
${mask(p.e)}`);
      if (newHash) await rpc("stars_member_set_password", { p_email: p.e,
p_hash: newHash });
      ensured.add(p.e);
    } catch (e) {
      console.error("[comptes] enregistrement impossible :", e.message);
      if (newHash) { a.used = false; return json(res, 503, { code:
"members_down" }); } // le code reste valable : on peut réessayer
    }
  }
  openSession(req, res, p.e, p.n);
  callMailer({ action: "verified", email: p.e, name: p.n }).catch(e =>
console.warn("[auth] note dans la Google Sheet impossible :", e.message));
  console.log(`[auth] ${p.m === "reset" ? "nouveau mot de passe et connexion"
: "connexion"} de ${mask(p.e)}`);
  json(res, 200, { email: p.e, name: p.n });
}

// ---------- Nettoyage de la conversation ----------
const IMAGE_TYPES = new Set(["image/jpeg", "image/png", "image/webp"]);
const MAX_IMAGES_PER_MSG = 3;
const MAX_IMAGES_TOTAL = 6;
const MAX_IMAGE_B64 = 3_500_000; // ≈ 2,6 Mo par image
const PHOTO_ONLY_TEXT = "Voici une photo. Si c'est un exercice, lis l'énoncé
et résous-le étape par étape.";

function cleanImages(list) {
  if (!Array.isArray(list)) return [];
  return list
    .filter(i => i && IMAGE_TYPES.has(i.mime) && typeof i.data === "string"
&& i.data.length > 100 && i.data.length <= MAX_IMAGE_B64 && /^[A-Za-z0-
9+/=]+$/.test(i.data))
    .slice(0, MAX_IMAGES_PER_MSG)
    .map(i => ({ mime: i.mime, data: i.data }));
}

function cleanMessages(raw) {
```

```
  if (!Array.isArray(raw)) return null;
  let msgs = raw
    .filter(m => m && (m.role === "user" || m.role === "assistant") && typeof
m.content === "string")
    .map(m => ({ role: m.role, content: m.content.slice(0, 12000), images:
m.role === "user" ? cleanImages(m.images) : [] }))
    .filter(m => m.content.trim() || m.images.length);
  msgs = msgs.slice(-20);
  while (msgs.length && msgs[0].role !== "user") msgs.shift();
  const merged = [];
  for (const m of msgs) {
    const last = merged[merged.length - 1];
    if (last && last.role === m.role) {
      last.content = [last.content, m.content].filter(s =>
s.trim()).join("\n\n");
      last.images.push(...m.images);
    } else merged.push({ ...m, images: [...m.images] });
  }
  if (!merged.length || merged[merged.length - 1].role !== "user") return
null;
  // Garde seulement les images les plus récentes.
  let budget = MAX_IMAGES_TOTAL;
  for (let i = merged.length - 1; i >= 0; i--) {
    const m = merged[i];
    if (m.images.length > budget) m.images = m.images.slice(m.images.length -
budget);
    budget -= m.images.length;
    if (!m.content.trim()) m.content = PHOTO_ONLY_TEXT;
  }
  const total = merged.reduce((n, m) => n + m.content.length, 0);
  if (total > 60000) return "too_large";
  return merged;
}

const toClaude = messages => messages.map(m => ({
  role: m.role,
  content: m.images.length
    ? [...m.images.map(i => ({ type: "image", source: { type: "base64",
media_type: i.mime, data: i.data } })), { type: "text", text: m.content }]
    : m.content
}));

const toGemini = messages => messages.map(m => ({
  role: m.role === "assistant" ? "model" : "user",
  parts: [...m.images.map(i => ({ inlineData: { mimeType: i.mime, data:
```

```
i.data } })), { text: m.content }]
}));

// ---------- Lecture d'un flux SSE ----------
async function* sse(body) {
  const decoder = new TextDecoder();
  let buf = "";
  for await (const chunk of body) {
    buf += decoder.decode(chunk, { stream: true });
    let i;
    while ((i = buf.indexOf("\n")) >= 0) {
      const line = buf.slice(0, i).replace(/\r$/, "");
      buf = buf.slice(i + 1);
      if (line.startsWith("data:")) {
        const data = line.slice(5).trim();
        if (data && data !== "[DONE]") { try { yield JSON.parse(data); }
catch {} }
      }
    }
  }
}

class UpstreamError extends Error {
  constructor(code, status, detail) { super(code); this.code = code;
this.status = status; this.detail = detail; }
}
function codeFor(status) {
  if (status === 401 || status === 403) return "bad_key";
  if (status === 429 || status === 503 || status === 529) return
"upstream_busy";
  if (status === 402) return "no_credit";
  return "upstream_error";
}
// Erreurs pour lesquelles on essaie le modèle suivant.
const RETRY_NEXT = new Set([404, 429, 500, 503, 529]);

// Un modèle qui a atteint sa limite (429), qui est surchargé (503) ou qui ne
répond pas est mis de côté
// un moment : les messages suivants vont directement au modèle qui marche,
sans attendre.
const modelPause = new Map(); // modèle -> pause jusqu'à (timestamp)
function pauseModel(model, ms, why) {
  modelPause.set(model, Date.now() + ms);
  console.warn(`[modèles] ${model} mis de côté ${Math.round(ms / 60000)} min
(${why})`);
```

```
}
function usableModels(list) {
  const now = Date.now();
  const ok = list.filter(m => (modelPause.get(m) || 0) <= now);
  return ok.length ? ok : list; // si tout est en pause, on réessaie quand
même
}
const PAUSE_FOR = { 429: 15 * 60_000, 404: 6 * 3600_000, 500: 60_000, 503: 2
* 60_000, 529: 2 * 60_000 };

// ---------- Claude (Anthropic) ----------
async function callClaude(messages, signal, name, voice, guide) {
  const models = usableModels([CLAUDE_MODEL, ...CLAUDE_FALLBACKS.filter(m =>
m !== CLAUDE_MODEL)]);
  let last = new UpstreamError("upstream_error", 404, "Aucun modèle Claude
disponible");
  for (const model of models) {
    let res;
    try {
      res = await fetch(`${ANTHROPIC_BASE_URL}/v1/messages`, {
        method: "POST",
        signal: withTimeout(signal, FIRST_BYTE_TIMEOUT_MS),
        headers: { "content-type": "application/json", "x-api-key":
ANTHROPIC_API_KEY, "anthropic-version": "2023-06-01" },
        body: JSON.stringify({ model, max_tokens: MAX_TOKENS, stream: true,
system: rules(name, voice, guide), messages: toClaude(messages) })
      });
    } catch (e) {
      if (signal.aborted) throw e;
      pauseModel(model, 5 * 60_000, "ne répond pas");
      last = new UpstreamError("upstream_busy", 504, "timeout");
      continue;
    }
    if (res.ok) return { res, model };
    const detail = await res.text().catch(() => "");
    const low = detail.toLowerCase();
    if (res.status === 400 && (low.includes("credit balance") ||
low.includes("billing"))) throw new UpstreamError("no_credit", res.status,
detail);
    if (RETRY_NEXT.has(res.status)) { pauseModel(model, PAUSE_FOR[res.status]
|| 60_000, "HTTP " + res.status); last = new
UpstreamError(codeFor(res.status), res.status, detail); continue; }
    throw new UpstreamError(codeFor(res.status), res.status, detail);
  }
  throw last;
```

```
}

async function* streamClaude(res) {
  for await (const ev of sse(res.body)) {
    if (ev.type === "content_block_delta" && ev.delta?.type === "text_delta")
yield { t: ev.delta.text };
    else if (ev.type === "message_delta" && ev.delta?.stop_reason ===
"max_tokens") yield { truncated: true };
    else if (ev.type === "message_delta" && ev.delta?.stop_reason ===
"refusal") yield { refused: true };
    else if (ev.type === "error") throw new UpstreamError(ev.error?.type ===
"overloaded_error" ? "upstream_busy" : "upstream_error", 500,
JSON.stringify(ev.error));
  }
}

// ---------- Gemini (Google) ----------
async function callGemini(messages, signal, name, voice, guide) {
  const models = usableModels([GEMINI_MODEL, ...GEMINI_FALLBACKS.filter(m =>
m !== GEMINI_MODEL)]);
  const contents = toGemini(messages);
  let last = new UpstreamError("upstream_error", 404, "Aucun modèle Gemini
disponible");
  for (const model of models) {
    let res;
    try {
      res = await
fetch(`${GEMINI_BASE_URL}/v1beta/models/${encodeURIComponent(model)}:streamGe
nerateContent?alt=sse`, {
        method: "POST",
        signal: withTimeout(signal, FIRST_BYTE_TIMEOUT_MS),
        headers: { "content-type": "application/json", "x-goog-api-key":
GEMINI_API_KEY },
        body: JSON.stringify({
          systemInstruction: { parts: [{ text: rules(name, voice, guide) }]
},
          contents,
          generationConfig: { maxOutputTokens: MAX_TOKENS }
        })
      });
    } catch (e) {
      if (signal.aborted) throw e;
      pauseModel(model, 5 * 60_000, "ne répond pas");
      last = new UpstreamError("upstream_busy", 504, "timeout");
      continue;
```

```
    }
    if (res.ok) return { res, model };
    const detail = await res.text().catch(() => "");
    if (res.status === 400 && /api key not
valid|api_key_invalid/i.test(detail)) throw new UpstreamError("bad_key", 401,
detail);
    if (RETRY_NEXT.has(res.status)) { pauseModel(model, PAUSE_FOR[res.status]
|| 60_000, "HTTP " + res.status); last = new
UpstreamError(codeFor(res.status), res.status, detail); continue; }
    throw new UpstreamError(codeFor(res.status), res.status, detail);
  }
  throw last;
}

async function* streamGemini(res) {
  for await (const ev of sse(res.body)) {
    if (ev.error) throw new UpstreamError("upstream_error", ev.error.code ||
500, JSON.stringify(ev.error));
    if (ev.promptFeedback?.blockReason) { yield { refused: true }; continue;
}
    const c = ev.candidates?.[0];
    if (!c) continue;
    for (const p of c.content?.parts || []) if (p.text && !p.thought) yield {
t: p.text };
    if (c.finishReason === "MAX_TOKENS") yield { truncated: true };
    else if (c.finishReason === "SAFETY" || c.finishReason ===
"PROHIBITED_CONTENT") yield { refused: true };
  }
}

// ---------- HTTP ----------
function json(res, status, obj) {
  res.writeHead(status, { "content-type": "application/json; charset=utf-8",
"cache-control": "no-store" });
  res.end(JSON.stringify(obj));
}

class HttpError extends Error { constructor(status, code) { super(code);
this.status = status; this.code = code; } }

async function readJson(req, limit = 200_000) {
  if (!String(req.headers["content-type"] ||
"").includes("application/json")) throw new HttpError(415, "bad_request");
  let size = 0; const chunks = [];
  for await (const c of req) { size += c.length; if (size > limit) throw new
```

```
HttpError(413, "too_large"); chunks.push(c); }
  try { return JSON.parse(Buffer.concat(chunks).toString("utf8")); } catch {
throw new HttpError(400, "bad_request"); }
}

function clientIp(req) {
  const fwd = req.headers["x-forwarded-for"];
  return (typeof fwd === "string" && fwd.split(",")[0].trim()) ||
req.socket.remoteAddress || "?";
}

async function handleChat(req, res, ip) {
  const user = currentUser(req);
  if (AUTH_REQUIRED && !user) return json(res, 401, { code: "auth_required"
});
  if (!provider) return json(res, 503, { code: "no_key" });

  const payload = await readJson(req, 16_000_000);
  const messages = cleanMessages(payload?.messages);
  if (messages === "too_large") return json(res, 413, { code: "too_large" });
  if (!messages) return json(res, 400, { code: "bad_request" });

  if (!allow("chat:" + (user ? user.email : ip), PER_MINUTE, 60_000)) return
json(res, 429, { code: "rate_limited" });
  // Compte gratuit : FREE_DAILY messages par jour ; Premium et propriétaire
(ADMIN_EMAILS) : illimité.
  const owner = isAdmin(user);
  let premium = owner;
  if (MEMBERS && user) {
    try {
      const rows = await rpc("stars_chat_allow", { p_email: user.email,
p_limit: owner ? 1_000_000 : FREE_DAILY });
      const r = Array.isArray(rows) ? rows[0] : null;
      if (r && !r.allowed) return json(res, 402, { code: "free_limit", limit:
FREE_DAILY, used: r.used, price: PREMIUM_PRICE });
      premium = owner || !!(r && r.premium_until && new Date(r.premium_until)
> new Date());
    } catch (e) { console.warn("[premium] compteur indisponible :",
e.message); }
  }
  const limited = checkSiteCap(premium);
  if (limited) return json(res, 429, { code: limited });

  const ac = new AbortController();
  res.on("close", () => { if (!res.writableEnded) ac.abort(); });
```

```
  let upstream;
  let prov = provider;
  const voice = payload?.voice === true;
  const guide = guideFor(messages);
  try {
    try {
      upstream = prov === "claude" ? await callClaude(messages, ac.signal,
user?.name, voice, guide) : await callGemini(messages, ac.signal, user?.name,
voice, guide);
    } catch (e) {
      // Secours : si Claude échoue (crédit épuisé, clé refusée, surcharge…)
et qu'une clé Gemini existe, on réessaie avec Gemini.
      if (ac.signal.aborted || prov !== "claude" || !GEMINI_API_KEY) throw e;
      console.error(`[secours] Claude a échoué (${e.code || e.message}),
bascule sur Gemini`);
      prov = "gemini";
      upstream = await callGemini(messages, ac.signal, user?.name, voice,
guide);
      console.log("[secours] Gemini utilisé");
    }
  } catch (e) {
    if (ac.signal.aborted) return;
    console.error(`[${prov}]`, e.code || e.message, e.status || "", (e.detail
|| "").slice(0, 300));
    return json(res, 502, { code: e.code || "upstream_error" });
  }

  res.writeHead(200, { "content-type": "application/x-ndjson; charset=utf-8",
"cache-control": "no-store", "x-accel-buffering": "no" });
  const send = obj => res.write(JSON.stringify(obj) + "\n");
  let truncated = false, refused = false, any = false;
  try {
    const gen = prov === "claude" ? streamClaude(upstream.res) :
streamGemini(upstream.res);
    for await (const ev of gen) {
      if (ev.t) { any = true; send({ t: ev.t }); }
      if (ev.truncated) truncated = true;
      if (ev.refused) refused = true;
    }
    if (refused && !any) send({ error: "refused" });
    else if (!any) send({ error: "empty_completion" });
    send({ done: true, truncated });
  } catch (e) {
    if (!ac.signal.aborted) {
```

```
      console.error(`[${prov}] flux`, e.code || e.message, (e.detail ||
"").slice(0, 300));
      send({ error: e.code || "upstream_error" });
    }
  } finally {
    res.end();
  }
}

let indexHtml = null;
async function page() {
  if (!indexHtml || process.env.NODE_ENV === "development") indexHtml = await
readFile(path.join(__dirname, "public", "index.html"));
  return indexHtml;
}

// ---------- Fichiers de l'appli (icônes, manifeste, service worker, pages)
----------
const PUBLIC_DIR = path.join(__dirname, "public");
const TYPES = { ".apk": "application/vnd.android.package-archive", ".png":
"image/png", ".jpg": "image/jpeg", ".svg": "image/svg+xml", ".webmanifest":
"application/manifest+json", ".js": "text/javascript; charset=utf-8",
".html": "text/html; charset=utf-8", ".json": "application/json" };
const PAGES = { "/antivirus": "antivirus.html", "/monscanner":
"antivirus.html", "/premium": "premium.html", "/admin": "admin.html",
"/ecoles": "ecoles.html", "/telecharger": "telecharger.html", "/download":
"telecharger.html", "/telecharger/stars-ai.apk": "downloads/stars-ai.apk",
"/confidentialite": "confidentialite.html", "/privacy":
"confidentialite.html", "/offline.html": "offline.html",
"/manifest.webmanifest": "manifest.webmanifest", "/sw.js": "sw.js" };
async function serveStatic(req, res, pathname) {
  let rel = PAGES[pathname];
  if (!rel && /^\/icons\/[a-z0-9-]+\.(png|jpg)$/.test(pathname)) rel =
pathname.slice(1);
  if (!rel) return false;
  const file = path.join(PUBLIC_DIR, rel);
  if (!file.startsWith(PUBLIC_DIR + path.sep)) return false;
  let data;
  try { data = await readFile(file); } catch { return false; }
  const ext = path.extname(file);
  const cache = rel.startsWith("icons/") ? "public, max-age=604800" : rel ===
"sw.js" ? "no-cache" : "public, max-age=3600";
  const headers = { "content-type": TYPES[ext] || "application/octet-stream",
"cache-control": cache, "content-length": data.length };
  if (ext === ".apk") headers["content-disposition"] = 'attachment;
```

```
filename="STARS-AI.apk"';
  res.writeHead(200, headers);
  res.end(req.method === "HEAD" ? undefined : data);
  return true;
}
// Lien entre l'appli Android et le site (rempli plus tard avec l'empreinte
donnée par PWABuilder / Google Play)
const ASSETLINKS_JSON = env("ASSETLINKS_JSON");

// ---------- Signalements et suppression de compte ----------
const REPORT_REASONS = new Set(["offensant", "faux_dangereux",
"inapproprie_jeunes", "autre"]);
async function handleReport(req, res, ip) {
  const user = currentUser(req);
  if (AUTH_REQUIRED && !user) return json(res, 401, { code: "auth_required"
});
  if (!MEMBERS) return json(res, 503, { code: "members_down" });
  const body = await readJson(req, 100_000);
  const reason = String(body?.reason || "");
  const answer = String(body?.answer || "").slice(0, 8000);
  if (!REPORT_REASONS.has(reason) || !answer.trim()) return json(res, 400, {
code: "bad_request" });
  if (!allow("report:" + (user ? user.email : ip), 15, 60 * 60_000)) return
json(res, 429, { code: "rate_limited" });
  try {
    await rpc("stars_report_add", {
      p_email: user ? user.email : "", p_reason: reason,
      p_comment: String(body?.comment || "").slice(0, 500), p_question:
String(body?.question || "").slice(0, 4000), p_answer: answer
    });
  } catch (e) { console.error("[signalement]", e.message); return json(res,
503, { code: "members_down" }); }
  console.log(`[signalement] ${reason} par ${user ? mask(user.email) :
"anonyme"}`);
  json(res, 200, { ok: true });
}

// ---------- Premium ----------
function premiumInfo(req, res) {
  const u = currentUser(req);
  if (!u) return json(res, 401, { code: "auth_required" });
  json(res, 200, { price: PREMIUM_PRICE, freeDaily: FREE_DAILY, methods:
PAY_METHODS, contactEmail: CONTACT_EMAIL, whatsapp: CONTACT_WHATSAPP || null
});
}
```

```
async function premiumRequest(req, res) {
  const u = currentUser(req);
  if (!u) return json(res, 401, { code: "auth_required" });
  if (!MEMBERS) return json(res, 503, { code: "members_down" });
  const body = await readJson(req);
  const method = String(body?.method || "");
  if (!PAY_METHODS.some(m => m.id === method)) return json(res, 400, { code:
"bad_method" });
  const phone = String(body?.phone || "").replace(/[^\d+ ]/g, "").trim();
  if (phone.replace(/\D/g, "").length < 8 || phone.length > 30) return
json(res, 400, { code: "bad_phone" });
  const reference = String(body?.reference || "").replace(/[\u0000-
\u001f<>]/g, "").trim().slice(0, 80);
  if (reference.replace(/\s/g, "").length < 6 || /^auto:/i.test(reference))
return json(res, 400, { code: "bad_reference" }); // la référence du SMS
prouve qu'un paiement a eu lieu
  if (!allow("payreq:" + u.email, 5, 60 * 60_000)) return json(res, 429, {
code: "rate_limited" });
  try {
    await rpc("stars_payment_add", { p_email: u.email, p_name: u.name || "",
p_method: method, p_phone: phone, p_reference: reference, p_amount:
PREMIUM_PRICE });
  } catch (e) {
    if (/too_many_pending/.test(e.message)) return json(res, 429, { code:
"too_many_pending" });
    console.error("[premium] demande impossible :", e.message);
    return json(res, 503, { code: "members_down" });
  }
  console.log(`[premium] demande de ${mask(u.email)} via ${method}`);
  notifyAdmins("paiement").catch(() => {});
  json(res, 200, { ok: true });
}

// ---------- Paiement Wave automatique (API Checkout) ----------
// 1. /api/premium/wave crée une session de paiement Wave et renvoie le lien
qui ouvre l'appli Wave.
// 2. Au retour sur /premium, /api/premium/wave/check demande à Wave si la
session est payée.
// 3. En plus, Wave prévient le serveur (webhook signé) même si la personne
ne revient pas sur la page.
// Chaque session Wave n'active le Premium qu'une seule fois (contrôlé dans
Supabase).
async function waveApi(method, pathname, body) {
  const r = await fetch(WAVE_API_URL + pathname, {
```

```
    method, signal: AbortSignal.timeout(15000),
    headers: { authorization: "Bearer " + WAVE_API_KEY, ...(body ? {
"content-type": "application/json" } : {}) },
    body: body ? JSON.stringify(body) : undefined
  });
  const text = await r.text();
  let data = null;
  try { data = text ? JSON.parse(text) : null; } catch {}
  if (!r.ok) {
    const e = new Error(`Wave ${method} ${pathname.split("?")[0]} HTTP
${r.status} : ${text.slice(0, 200).replace(/\s+/g, " ")}`);
    e.status = r.status;
    throw e;
  }
  return data;
}

// La référence envoyée à Wave est chiffrée : Wave ne voit pas l'e-mail, et
personne ne peut la fabriquer.
const waveRef = email => seal(JSON.stringify({ k: "wave", e: email }));
function waveOwner(s) {
  try {
    const ref = JSON.parse(unseal(String(s?.client_reference || "")) ||
"null");
    return ref && ref.k === "wave" && typeof ref.e === "string" ? ref.e :
null;
  } catch { return null; }
}
const waveLast = new Map(); // dernière session Wave de chaque personne (si
la page de retour s'ouvre dans un autre navigateur)

function publicOrigin(req) {
  const fixed = env("PUBLIC_URL") || env("RENDER_EXTERNAL_URL");
  if (fixed) return fixed.replace(/\/+$/, "");
  const proto = String(req.headers["x-forwarded-proto"] || "").split(",")
[0].trim() || "http";
  return `${proto}://${req.headers.host}`;
}

// Active le Premium si la session Wave (lue chez Wave ou reçue par webhook
signé) est payée.
async function waveSettle(s, source) {
  const status = s?.payment_status || "unknown";
  if (status !== "succeeded") return { status, checkout: s?.checkout_status
|| null };
```

```
  const email = waveOwner(s);
  if (!email) { console.warn(`[wave] paiement ${s.id} sans référence STARS AI
: ignoré`); return { status: "unknown_ref" }; }
  if (s.currency !== "XOF" || !(Number(s.amount) >= PREMIUM_PRICE)) {
    console.warn(`[wave] paiement ${s.id} : montant inattendu ${s.amount}
${s.currency}`);
    return { status: "bad_amount" };
  }
  const row = (await rpc("stars_wave_paid", { p_email: email, p_name: "",
p_session: String(s.id), p_amount: Math.round(Number(s.amount)), p_phone: ""
}))?.[0];
  if (row && !row.already) console.log(`[wave] Premium activé automatiquement
pour ${mask(email)} (${source})`);
  return { status: "succeeded", email, premiumUntil: row?.premium_until ||
null };
}

async function waveStart(req, res) {
  const u = currentUser(req);
  if (!u) return json(res, 401, { code: "auth_required" });
  await readJson(req);
  if (!WAVE_AUTO) return json(res, 404, { code: "wave_off" });
  if (!allow("wave:" + u.email, 10, 60 * 60_000)) return json(res, 429, {
code: "rate_limited" });
  const reference = waveRef(u.email);
  if (reference.length > 255) return json(res, 400, { code: "wave_off" });
  const origin = publicOrigin(req);
  try {
    const s = await waveApi("POST", "/v1/checkout/sessions", {
      amount: String(PREMIUM_PRICE), currency: "XOF",
      success_url: origin + "/premium?wave=ok", error_url: origin +
"/premium?wave=erreur",
      client_reference: reference
    });
    if (!s?.id || !/^https:\/\//.test(String(s.wave_launch_url || ""))) throw
new Error("réponse Wave incomplète");
    waveLast.set(u.email, s.id);
    console.log(`[wave] paiement lancé pour ${mask(u.email)}`);
    json(res, 200, { id: s.id, url: s.wave_launch_url });
  } catch (e) {
    console.error("[wave] création du paiement impossible :", e.message);
    json(res, 502, { code: "wave_down" });
  }
}
```

```
async function waveCheck(req, res) {
  const u = currentUser(req);
  if (!u) return json(res, 401, { code: "auth_required" });
  const b = await readJson(req);
  if (!WAVE_AUTO) return json(res, 404, { code: "wave_off" });
  const id = String(b?.id || waveLast.get(u.email) || "");
  if (!/^cos-[A-Za-z0-9_-]{4,64}$/.test(id)) return json(res, 404, { code:
"no_session" });
  if (!allow("wavecheck:" + u.email, 120, 60 * 60_000)) return json(res, 429,
{ code: "rate_limited" });
  try {
    const s = await waveApi("GET", "/v1/checkout/sessions/" +
encodeURIComponent(id));
    if (waveOwner(s) !== u.email) return json(res, 404, { code: "no_session"
});
    const out = await waveSettle(s, "retour sur la page");
    json(res, 200, { status: out.status, checkout: s.checkout_status || null,
premiumUntil: out.premiumUntil || null });
  } catch (e) {
    if (e.status === 404) return json(res, 404, { code: "no_session" });
    console.error("[wave] vérification impossible :", e.message);
    json(res, 502, { code: "wave_down" });
  }
}

// Webhook Wave (Business Portal > Développeurs > Webhooks) : en-tête Wave-
Signature « t=…,v1=… »,
// HMAC-SHA256 de (t + corps brut) avec le secret du webhook. On accepte
aussi la méthode « secret partagé ».
function waveSignatureOk(raw, header, auth) {
  const secret = WAVE_WEBHOOK_SECRET;
  if (header) {
    const parts = String(header).split(",").map(p => p.trim().split("="));
    const t = parts.find(p => p[0] === "t")?.[1];
    const sigs = parts.filter(p => p[0] === "v1" && p[1]).map(p => p[1]);
    if (!t || !/^\d+$/.test(t) || !sigs.length || Math.abs(Date.now() / 1000
- Number(t)) > 300) return false;
    const expected = crypto.createHmac("sha256",
secret).update(t).update(raw).digest();
    return sigs.some(s => { const got = Buffer.from(s, "hex"); return
got.length === expected.length && crypto.timingSafeEqual(got, expected); });
  }
  const m = /^Bearer (.+)$/.exec(String(auth || ""));
  if (!m) return false;
  const a = crypto.createHash("sha256").update(m[1]).digest(), e =
```

```
crypto.createHash("sha256").update(secret).digest();
  return crypto.timingSafeEqual(a, e);
}

async function waveWebhook(req, res) {
  let size = 0; const chunks = [];
  for await (const c of req) { size += c.length; if (size > 100_000) return
json(res, 413, { code: "too_large" }); chunks.push(c); }
  if (!WAVE_WEBHOOK_SECRET || !WAVE_AUTO) return json(res, 404, { code:
"wave_off" });
  const raw = Buffer.concat(chunks);
  if (!waveSignatureOk(raw, req.headers["wave-signature"],
req.headers.authorization)) {
    console.warn("[wave] webhook refusé : signature invalide");
    return json(res, 401, { code: "bad_signature" });
  }
  let ev;
  try { ev = JSON.parse(raw.toString("utf8")); } catch { return json(res,
400, { code: "bad_request" }); }
  try {
    if (ev?.type === "checkout.session.completed") await waveSettle(ev.data,
"webhook");
    else if (ev?.type === "test.test_event") console.log("[wave] webhook de
test reçu ✓");
  } catch (e) {
    console.error("[wave] webhook non traité (Wave réessaiera) :",
e.message);
    return json(res, 503, { code: "retry" });
  }
  json(res, 200, { ok: true });
}

// ---------- MonScanner en ligne (antivirus de vérification, par Yvan Mambo)
----------
// Le navigateur calcule l'empreinte SHA-256 du fichier : le fichier ne
quitte jamais l'appareil.
// Le serveur demande seulement à VirusTotal ce qu'il sait de cette empreinte
(clé VT_API_KEY, compte gratuit).
const VT_API_KEY = env("VT_API_KEY");
const VT_BASE_URL = (env("VT_BASE_URL") ||
"https://www.virustotal.com").replace(/\/+$/, "");
const VT_PER_MINUTE = Number(env("VT_PER_MINUTE")) || 4; // limite du compte
gratuit VirusTotal
const VT_PER_DAY = Number(env("VT_PER_DAY")) || 480;     // 500 par jour : on
garde une marge
```

```
const VT_CACHE_MS = 6 * 3600_000;
const vtCalls = [];
let vtDay = "", vtDayCount = 0;
const vtCache = new Map(); // empreinte -> { t, data } (les fichiers déjà
vérifiés ne coûtent rien)

function vtBudget() {
  const now = Date.now(), day = new Date(now).toISOString().slice(0, 10);
  if (day !== vtDay) { vtDay = day; vtDayCount = 0; }
  while (vtCalls.length && now - vtCalls[0] > 60_000) vtCalls.shift();
  if (vtDayCount >= VT_PER_DAY) {
    const midnight = Date.UTC(new Date(now).getUTCFullYear(), new
Date(now).getUTCMonth(), new Date(now).getUTCDate() + 1);
    return { ok: false, daily: true, wait: Math.ceil((midnight - now) / 1000)
};
  }
  if (vtCalls.length >= VT_PER_MINUTE) return { ok: false, wait:
Math.ceil((60_000 - (now - vtCalls[0])) / 1000) + 1 };
  vtCalls.push(now); vtDayCount++;
  return { ok: true };
}
function vtRemember(sha, data) {
  vtCache.set(sha, { t: Date.now(), data });
  if (vtCache.size > 5000) vtCache.delete(vtCache.keys().next().value);
}

async function handleScan(req, res, ip) {
  const body = await readJson(req, 2000);
  const sha = String(body?.sha256 || "").toLowerCase();
  if (!/^[0-9a-f]{64}$/.test(sha)) return json(res, 400, { code: "bad_hash"
});
  if (!VT_API_KEY) return json(res, 503, { code: "vt_off" });
  const hit = vtCache.get(sha);
  if (hit && Date.now() - hit.t < VT_CACHE_MS) return json(res, 200,
hit.data);
  if (!allow("scan:" + ip, 40, 60 * 60_000)) return json(res, 429, { code:
"rate_limited", retryAfter: 900 });
  const budget = vtBudget();
  if (!budget.ok) return json(res, 429, { code: budget.daily ? "vt_daily" :
"vt_busy", retryAfter: budget.wait });
  let r;
  try {
    r = await fetch(`${VT_BASE_URL}/api/v3/files/${sha}`, { headers: { "x-
apikey": VT_API_KEY, accept: "application/json" }, signal:
AbortSignal.timeout(20000) });
```

```
  } catch (e) {
    console.error("[antivirus] VirusTotal injoignable :", e.message);
    return json(res, 502, { code: "vt_down" });
  }
  if (r.status === 404) { const data = { known: false }; vtRemember(sha,
data); return json(res, 200, data); }
  if (r.status === 429) return json(res, 429, { code: "vt_busy", retryAfter:
60 });
  if (!r.ok) {
    console.error(`[antivirus] VirusTotal a répondu HTTP ${r.status} :
${(await r.text().catch(() => "")).slice(0, 200)}`);
    return json(res, 502, { code: r.status === 401 || r.status === 403 ?
"vt_key" : "vt_down" });
  }
  const a = (await r.json().catch(() => null))?.data?.attributes || {};
  const s = a.last_analysis_stats || {};
  const n = k => Number(s[k]) || 0;
  const data = {
    known: true,
    malicious: n("malicious"), suspicious: n("suspicious"),
    total: n("malicious") + n("suspicious") + n("undetected") +
n("harmless"),
    label: a.popular_threat_classification?.suggested_threat_label || null,
    type: a.type_description || null,
    analysed: a.last_analysis_date ? new Date(a.last_analysis_date *
1000).toISOString() : null,
    // Les moteurs antivirus qui le détectent, avec le nom de la menace
(comme dans un vrai rapport d'analyse)
    detections: Object.entries(a.last_analysis_results || {})
      .filter(([, v]) => v && (v.category === "malicious" || v.category ===
"suspicious"))
      .sort((x, y) => (x[1].category === "malicious" ? 0 : 1) -
(y[1].category === "malicious" ? 0 : 1))
      .slice(0, 12)
      .map(([engine, v]) => ({ engine: String(v.engine_name ||
engine).slice(0, 40), result: String(v.result || (v.category === "malicious"
? "malveillant" : "suspect")).slice(0, 80) }))
  };
  vtRemember(sha, data);
  console.log(`[antivirus] fichier vérifié : ${data.malicious}/${data.total}
antivirus le signalent`);
  json(res, 200, data);
}

// ---------- Tableau de bord du propriétaire ----------
```

```
async function adminApi(req, res, url) {
  const u = currentUser(req);
  if (!u) return json(res, 401, { code: "auth_required" });
  if (!isAdmin(u)) return json(res, 403, { code: "forbidden" });
  if (!MEMBERS) return json(res, 503, { code: "members_down" });
  try {
    if (req.method === "GET" && url.pathname === "/api/admin/overview") {
      return json(res, 200, { ...(await rpc("stars_admin_overview", {})),
price: PREMIUM_PRICE, freeDaily: FREE_DAILY });
    }
    if (req.method === "POST" && url.pathname === "/api/admin/payment") {
      const b = await readJson(req);
      const rows = await rpc("stars_payment_decide", { p_id: Number(b?.id),
p_approve: b?.approve === true });
      console.log(`[admin] paiement ${b?.id} ${b?.approve === true ? "validé"
: "refusé"}`);
      return json(res, 200, { ok: true, premiumUntil: rows?.
[0]?.premium_until || null });
    }
    if (req.method === "POST" && url.pathname === "/api/admin/grant") {
      const b = await readJson(req);
      const until = await rpc("stars_premium_grant", { p_email:
String(b?.email || "").trim().toLowerCase(), p_months: Number(b?.months) || 1
});
      return json(res, 200, { ok: true, premiumUntil: until });
    }
    if (req.method === "POST" && url.pathname === "/api/admin/push-
subscribe") {
      const b = await readJson(req);
      const endpoint = String(b?.endpoint || "");
      if (!(await loadVapid())) return json(res, 503, { code: "push_off" });
      if (!pushEndpointOk(endpoint) || endpoint.length > 1000) return
json(res, 400, { code: "bad_endpoint" });
      await rpc("stars_push_save", { p_email: u.email, p_endpoint: endpoint
});
      return json(res, 200, { ok: true });
    }
    if (req.method === "POST" && url.pathname === "/api/admin/push-test") {
      await readJson(req);
      await notifyAdmins("test");
      return json(res, 200, { ok: true });
    }
    if (req.method === "POST" && url.pathname === "/api/admin/report") {
      const b = await readJson(req);
      await rpc("stars_report_mark", { p_id: Number(b?.id), p_status:
```

```
b?.status === "traite" ? "traite" : "vu" });
      return json(res, 200, { ok: true });
    }
  } catch (e) {
    if (e instanceof HttpError) throw e;
    const known =
/not_pending|not_registered|bad_months|bad_status/.exec(e.message);
    if (!known) console.error("[admin]", e.message);
    return json(res, known ? 400 : 503, { code: known ? known[0] :
"members_down" });
  }
  json(res, 404, { code: "not_found" });
}

async function handleDeleteAccount(req, res) {
  const user = currentUser(req);
  if (!user) return json(res, 401, { code: "auth_required" });
  await readJson(req); // exige une requête JSON (protège contre les
formulaires d'autres sites)
  if (MEMBERS) {
    try { await rpc("stars_member_remove", { p_email: user.email }); }
    catch (e) { console.error("[comptes] suppression impossible :",
e.message); return json(res, 503, { code: "members_down" }); }
  }
  ensured.delete(user.email);
  callMailer({ action: "deleted", email: user.email, name: user.name
}).catch(() => {}); // pour un futur script qui retire la ligne
  res.setHeader("set-cookie", sessionCookie(req, "", 0));
  console.log(`[comptes] compte supprimé : ${mask(user.email)}`);
  json(res, 200, { ok: true });
}

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, "http://x");
  const ip = clientIp(req);
  try {
    if (req.method === "GET" && (url.pathname === "/" || url.pathname ===
"/index.html")) {
      res.writeHead(200, { "content-type": "text/html; charset=utf-8",
"cache-control": "no-cache" });
      return res.end(await page());
    }
    if (req.method === "GET" && url.pathname === "/healthz") return json(res,
200, { ok: true });
    if ((req.method === "GET" || req.method === "HEAD") && await
```

```
serveStatic(req, res, url.pathname)) return;
    if (req.method === "GET" && url.pathname === "/.well-
known/assetlinks.json") {
      res.writeHead(ASSETLINKS_JSON ? 200 : 404, { "content-type":
"application/json", "cache-control": "public, max-age=300" });
      return res.end(ASSETLINKS_JSON || "[]");
    }
    if (req.method === "POST" && url.pathname === "/api/report") return await
handleReport(req, res, ip);
    if (req.method === "POST" && url.pathname === "/api/account/delete")
return await handleDeleteAccount(req, res);
    if (req.method === "GET" && url.pathname === "/api/status") return
json(res, 200, { ready: !!provider, provider, authRequired: AUTH_REQUIRED,
passwords: MEMBERS, passwordMin: PASSWORD_MIN });
    if (req.method === "GET" && url.pathname === "/api/me") {
      const u = currentUser(req);
      if (!u) return json(res, 401, { code: "auth_required" });
      ensureMember(u);
      const out = { ...u, admin: isAdmin(u), freeDaily: FREE_DAILY, price:
PREMIUM_PRICE };
      if (MEMBERS) {
        try {
          const r = (await rpc("stars_me_status", { p_email: u.email }))?.
[0];
          if (r) { out.used = r.used; out.premiumUntil = r.premium_until;
out.pending = r.pending; }
          if (out.admin) out.adminPending = await rpc("stars_admin_pending",
{});
        } catch (e) { console.warn("[premium]", e.message); }
      }
      return json(res, 200, out);
    }
    if (req.method === "GET" && url.pathname === "/api/push/key") {
      const ready = await loadVapid();
      return json(res, ready ? 200 : 503, { publicKey: ready ? VAPID_PUBLIC :
null });
    }
    if (req.method === "GET" && url.pathname === "/api/premium/info") return
premiumInfo(req, res);
    if (req.method === "POST" && url.pathname === "/api/premium/request")
return await premiumRequest(req, res);
    if (req.method === "POST" && url.pathname === "/api/premium/wave") return
await waveStart(req, res);
    if (req.method === "POST" && url.pathname === "/api/premium/wave/check")
return await waveCheck(req, res);
```

```
    if (req.method === "POST" && url.pathname === "/api/wave/webhook") return
await waveWebhook(req, res);
    if (req.method === "GET" && url.pathname === "/api/scan/status") return
json(res, 200, { virustotal: !!VT_API_KEY });
    if (req.method === "POST" && url.pathname === "/api/scan") return await
handleScan(req, res, ip);
    if (url.pathname.startsWith("/api/admin/")) return await adminApi(req,
res, url);
    if (req.method === "GET" && url.pathname === "/api/contact") {
      return json(res, 200, { email: CONTACT_EMAIL, whatsapp:
CONTACT_WHATSAPP || null, price: PREMIUM_PRICE, freeDaily: FREE_DAILY });
    }
    if (req.method === "POST" && url.pathname === "/api/auth/start") return
await authStart(req, res, ip);
    if (req.method === "POST" && url.pathname === "/api/auth/verify") return
await authVerify(req, res, ip);
    if (req.method === "POST" && url.pathname === "/api/auth/password")
return await authPassword(req, res, ip);
    if (req.method === "POST" && url.pathname === "/api/auth/logout") {
      res.setHeader("set-cookie", sessionCookie(req, "", 0));
      return json(res, 200, { ok: true });
    }
    if (req.method === "POST" && url.pathname === "/api/chat") return await
handleChat(req, res, ip);
    json(res, 404, { code: "not_found" });
  } catch (e) {
    if (e instanceof HttpError) return res.headersSent ? res.end() :
json(res, e.status, { code: e.code });
    console.error(e);
    if (!res.headersSent) json(res, 500, { code: "server_error" }); else
res.end();
  }
});

// ---------- Vérifications au démarrage (résultat dans les journaux Render,
sans afficher les clés) ----------
async function selfCheck() {
  try {
    if (provider === "gemini") {
      const r = await fetch(`${GEMINI_BASE_URL}/v1beta/models?pageSize=200`,
{ headers: { "x-goog-api-key": GEMINI_API_KEY }, signal:
AbortSignal.timeout(20000) });
      const body = await r.text();
      if (!r.ok) console.error(`[vérif] Clé Gemini REFUSÉE (HTTP ${r.status})
: ${body.slice(0, 300)}`);
```

```
      else {
        const names = (JSON.parse(body).models || []).map(m =>
m.name.replace("models/", ""));
        console.log(`[vérif] Clé Gemini OK — ${names.length} modèles
disponibles.`);
        for (const m of [GEMINI_MODEL, ...GEMINI_FALLBACKS]) if
(!names.includes(m)) pauseModel(m, 6 * 3600_000, "absent de la liste
Google");
      }
    } else if (provider === "claude") {
      const r = await fetch(`${ANTHROPIC_BASE_URL}/v1/models`, { headers: {
"x-api-key": ANTHROPIC_API_KEY, "anthropic-version": "2023-06-01" }, signal:
AbortSignal.timeout(20000) });
      console.log(r.ok ? "[vérif] Clé Claude OK." : `[vérif] Clé Claude
REFUSÉE (HTTP ${r.status}).`);
    }
  } catch (e) {
    console.error("[vérif] Impossible de joindre l'IA :", e.message);
  }
  if (MEMBERS) {
    try {
      await findMember("verification@example.com");
      console.log("[vérif] Base des comptes Supabase OK.");
    } catch (e) {
      console.error("[vérif] Base des comptes Supabase injoignable :",
e.message);
    }
  }
  if (MEMBERS) console.log(await loadVapid() ? "[vérif] Clés des
notifications OK." : "[vérif] Clés des notifications indisponibles.");
  if (MAIL_WEBHOOK_URL) {
    try {
      const d = await callMailer({ action: "ping" });
      console.log(`[vérif] Script Google OK — e-mails encore disponibles
aujourd'hui : ${d.quota}`);
    } catch (e) {
      console.error("[vérif] Script Google injoignable :", e.message);
    }
  }
  if (VT_API_KEY) {
    // Fichier de test EICAR (inoffensif, reconnu par tous les antivirus) :
vérifie la clé VirusTotal.
    try {
      const r = await
fetch(`${VT_BASE_URL}/api/v3/files/275a021bbfb6489e54d471899f7db9d1663fc695ec
```

```
2fe2a2c4538aabf651fd0f`, { headers: { "x-apikey": VT_API_KEY }, signal:
AbortSignal.timeout(20000) });
      vtBudget();
      console.log(r.ok ? "[vérif] Clé VirusTotal OK — MonScanner en ligne
actif." : `[vérif] Clé VirusTotal REFUSÉE (HTTP ${r.status}).`);
    } catch (e) { console.error("[vérif] VirusTotal injoignable :",
e.message); }
  }
  if (WAVE_API_KEY && !MEMBERS) console.error("[vérif] Clé Wave présente mais
base des comptes absente : paiement Wave automatique désactivé.");
  if (WAVE_AUTO) {
    try {
      await waveApi("GET", "/v1/checkout/sessions/search?
client_reference=verification");
      console.log(`[vérif] Clé Wave OK — paiement Wave automatique
actif${WAVE_WEBHOOK_SECRET ? " (avec webhook)" : " (sans webhook : activation
au retour sur la page)"}.`);
    } catch (e) {
      console.error(`[vérif] Clé Wave ${e.status === 401 || e.status === 403
? "REFUSÉE" : "injoignable"} :`, e.message);
    }
  }
}

server.listen(PORT, () => {
  console.log(`STARS AI en ligne sur le port ${PORT} — IA : ${provider ?
provider + " (" + (provider === "claude" ? CLAUDE_MODEL : GEMINI_MODEL) + ")"
: "AUCUNE CLÉ"} — inscription : ${AUTH_REQUIRED ? "obligatoire" :
"désactivée"}`);
  if (process.env.SKIP_SELF_CHECK !== "1") selfCheck();
});
```